FlexiSpace New Version Handover

Status date: 19 October 2026

Purpose: Record the work completed on the new FlexiSpace applications, the current integration constraints, and the work still needed before final delivery.

Project overview

The new version consists of an ASP.NET Core Web API, a Blazor web application, and a .NET MAUI mobile application. During local development, the web and mobile clients are configured to call the API running at `https://localhost:7055`. The web app's default non-development configuration still points to the hosted API, so the target must be checked before publishing or demonstrating the apps.

The project team has five members. Application roles are stored in FlexiSpace's `Users` table; Entra ID authenticates the identity, while the API uses the local role and location assignment to control application access.

Work completed in the new version

API, database, and blocked rooms

- Fixed the API compile blockers caused by duplicate DTO definitions. The API, web app, and their referenced projects now build.
- Resolved the blocked-period database migration mismatch. The migration `20261018183000_AlignBlockedPeriodsSchema` aligns the EF Core model with the existing `BlockedPeriods` table in the shared database.
- Admins can create blocked periods, and a blocked room is excluded through the room selection and booking flow. The user confirmed this works after applying the database update with `FlexiSpace.Infrastructure` selected as the migrations project.
- The database is shared with the team. Do not drop or recreate it to resolve migration problems; inspect the migration history and schema first.

Booking and availability

- Booking creation validates the room, booking time, capacity, conflicts, and selected active equipment and catering items.
- The client removed the booking approval requirement. New bookings are confirmed immediately in the API. Older `Pending`/approval types or UI code may remain in the repository and should not be presented as an active client requirement without review.
- The booking details step distinguishes equipment already included in the room from optional additional equipment. It also presents the room capacity and checks attendee counts before submission.
- Equipment and catering catalogue items that are inactive appear disabled in the booking flow. Active items can be selected. Admin catalogue changes are reflected in booking options.
- The user confirmed that blocked-room periods appear throughout the booking process and that the catalogue updates work.

Live availability status bar

- The web app's `/live` page displays counts for Available, Occupied, Reserved, Cleaning, Maintenance, and Blocked rooms.
- On 19 October, the Cleaning count was corrected. The API previously had no distinct `Cleaning` boardroom status, so the web app's Cleaning state could be collapsed into `Unavailable` and read back as Blocked.
- Cleaning now has API enum value 3; existing enum values remain unchanged (`Available = 0`, `Maintenance = 1`, `Unavailable = 2`). The web app maps the value both ways and keeps Cleaning from being overwritten during live availability calculation. This is an enum value change and does not require a database schema migration.
- The web project built with 0 warnings and 0 errors. The API built with 0 errors and 2 nullable-reference warnings in `BookingService.cs` (lines 146 and 360 at the time of the build). Restart the API and web app after pulling/building this change. If a room was previously saved as `Unavailable`, edit it and save it as Cleaning so its stored value is updated to 3.

Admin room and catalogue management

- Admins can add, edit, disable, and delete equipment and catering catalogue entries, subject to booking/history restrictions. Disabling an item is the safer choice when it is temporarily unavailable or already referenced by bookings.
- The booking form presents unavailable catalogue entries as greyed out and shows room-provided equipment separately from optional add-ons.
- The seeded location name is **Houghton Estate**. The source seed data uses this canonical name. Check the shared database for any old, separate `Houghton` record; correcting seed data does not automatically remove a legacy database row that is already referenced by rooms, users, or bookings.

Admin audit history

- The Admin Audit page records booking creation, updates, cancellations and status changes; user and blocked-period changes; and changes to locations, calendar accounts, boardrooms, equipment, and catering.
- Sign-in and sign-out audit events are supported through the session-audit API endpoints. The API resolves the acting user from the authenticated request.
- Audit entries include the actor, action, affected record, time, and relevant change details. Passwords and tokens are not included in change details. Audit write failures are intended not to fail the underlying business action.
- The user confirmed testing the Audit page changes.

Admin user management on web and mobile

- Both apps provide Provisioned and Unprovisioned views with totals. The web view uses tabs rather than a long combined list; the mobile view also uses tabs.
- The workflow is to create the identity in Microsoft Entra, find it under Unprovisioned, assign the FlexiSpace location, and provision it into the local `Users` table.
- The in-app option to create Entra users was removed. Administrators manage Entra account activation in Entra; the app's user edit workflow is limited to location assignment.
- Role and location filtering is present in the mobile user list. The mobile list deduplicates repeated Entra results by identity/email.

- The web API client was updated to accept role values returned as enum names as well as numbers, which addressed the JSON conversion error while loading Entra users.
- Final verification should confirm the same provisioned/unprovisioned totals and results in both apps, including a complete provision action. The final counts have not been recorded in this handover.

Email, Outlook, and identity constraints

Email

- The API registers the Microsoft Graph email service only when `MicrosoftGraph:TenantId`, `MicrosoftGraph:ClientId`, `MicrosoftGraph:ClientSecret`, and `MicrosoftGraph:SenderEmail` are configured. Otherwise it uses `NoOpEmailService`, so booking can still succeed without sending email.
- `BookingService.CreateBookingAsync` currently calls the booking confirmation email method on booking creation, on a best-effort basis. Email failure does not fail the booking.
- The email interface contains modification and cancellation methods, but the current booking update and cancellation methods do not call them. Connecting and verifying those messages remains outstanding for the new version.
- The sender mailbox is configured in `MicrosoftGraph:SenderEmail` in API configuration/user secrets for local development, or in the deployed API's secure configuration. It must be a mailbox that the client's tenant and Graph permissions allow the application to send as.
- Do not claim that email delivery works in the current environment unless Graph credentials, sender mailbox, permissions, and a successful end-to-end delivery have been verified.

Outlook calendar

- `LocationCalendarAccounts` stores calendar email, display name, primary/active flags, creation time, and location ID. There is no role column in this table; a calendar account is associated to a location, not assigned an application role.
- Graph calendar service code and location calendar account management exist. However, the create-event and availability-check calls inside the current `BookingService` booking flow are commented out. Booking updates and cancellations also do not currently synchronize calendar events in that flow.
- Multiple managers per location are represented by multiple active `LocationCalendarAccounts` rows, but sending or updating an event in every manager's calendar is not currently wired into the booking flow. Per-calendar event tracking should be reviewed before implementing multi-calendar update/cancellation behavior.
- Outlook integration remains unverified because the current test tenant does not provide the required manager Microsoft 365 mailboxes and Graph configuration. Keep the local FlexiSpace booking rules operational without Graph; enable and test calendar sync only after the client supplies mailboxes and grants the necessary permissions.

Entra ID and customer registration

- The web and mobile apps use Microsoft Entra sign-in, with FlexiSpace role and location information coming from the local `Users` record.
- A separate Microsoft Entra External ID customer tenant and customer self-registration were considered but deferred because of subscription/resource constraints. No customer tenant was created in this work.
- The app's policy/guest browsing flow remains an application feature. Customer self-registration and customer password-reset setup are not completed by the deferred External ID work.

Project documentation progress

- The Expo presentation guide was prepared as a plain, slide-by-slide demonstration guide, with topics, explanations, and evidence to show. The Expo should be presented primarily as a demonstration rather than reading dense slides.
- The User Guide is still outstanding because it needs screenshots from the current, updated applications.
- Task 1 documentation was corrected earlier. It should be checked once more against the final application changes before the final report is submitted.

Outstanding work before final delivery

- Verify the final user provisioning flow and matching counts on both web and mobile.
- Complete email handling for booking modification and cancellation, then verify all email events with the client's Microsoft Graph configuration.
- Re-enable and complete Outlook event creation, update, and deletion in the booking flow. Support every active calendar account at a location if the client requires events in multiple managers' calendars, and persist an event reference per target calendar.
- Confirm no legacy `Houghton` location remains as a separate record in the shared database; preserve the canonical Houghton Estate record and its history.
- Capture current screenshots and complete the User Guide.
- Reconcile the final report and Expo presentation with the delivered functionality and client plan.
- Publish the reviewed API version and confirm the web and mobile app configurations point to the intended deployed API.
- Complete end-to-end acceptance checks using the client's accounts, mailboxes, and hosting configuration.

Key files

- API startup and Graph service selection: `FlexiSpace/FlexiSpace.API/Program.cs`
- Booking validations, email confirmation call, and currently disabled Graph booking calls: `FlexiSpace/FlexiSpace.Infrastructure/services/BookingService.cs`
- EF Core schema changes: `FlexiSpace/FlexiSpace.Infrastructure/Migrations/`
- Location calendar account entity/configuration: `FlexiSpace/FlexiSpace.Core/Entities/LocationCalendarAccount.cs` and `FlexiSpace/FlexiSpace.Infrastructure/Persistence/Configurations/LocationCalendarAccountConfiguration.cs`
- Live availability page: `Flexispace.Web/FlexiSpace.Web/Components/Pages/Live.razor`
- Live availability counts: `Flexispace.Web/FlexiSpace.Web/ViewModels/BookingAndAvailabilityViewModels.cs`
- Cleaning status mapping: `FlexiSpace/FlexiSpace.Core/Enums/BoardroomStatus.cs`, `Flexispace.Web/FlexiSpace.Web/Api/ApiAdminService.cs`, `ApiEnumIntConverter.cs`, and `CatalogMapper.cs`

- Web user management: `Flexispace.Web/FlexiSpace.Web/Components/Pages/Users.razor` and `Flexispace.Web/FlexiSpace.Web/ViewModels/OtherViewModels.cs`
- Mobile user management: `Flexispace.Mobile/Flexispace.Mobile/Views/UsersPage.xaml` and `Flexispace.Mobile/Flexispace.Mobile/ViewModels/UsersViewModel.cs`
- Admin audit page: `Flexispace.Web/FlexiSpace.Web/Components/Pages/Audit.razor`
- API and client setup: `README.md`, `docs/Connect_Client_To_API.md`, `Flexispace.Mobile/Flexispace.Mobile/Services/Real/ApiConfig.cs`, and `Flexispace.Web/FlexiSpace.Web/appsettings.Development.json`